Generador de Carátulas Cutres con LLM

https://gemini.google.com/app/907ca419b92dc8df

User prompt: Vale pon que estoy haciendo un lector de comics novelas archivos CBR creo que Claude por ejemplo utiliza vectores para dibujar sería gracioso que para diseñar carátulas o los olace holder utilzasemos un llm para hacerlo así un poco cutre y gracioso como si fuese el Paint sabes se podría entrenar un modelo o no se investiga a ver

Response: La idea es genial porque aprovecha justo lo que peor se les da a los modelos de texto (la precisión espacial compleja) y lo convierte en una *feature* estética: ese estilo "hecho en MS Paint con el ratón en 2 minutos" que los LLMs clavan de forma natural cuando generan gráficos vectoriales (**SVG**).

No hace falta siquiera entrenar un modelo desde cero; con los modelos actuales y un par de trucos de arquitectura puedes montarlo de forma muy ligera para tu lector de CBR/CBZ.

1. Cómo "dibujan" los LLMs (El truco del SVG)

Cuando ves a Claude o a otros modelos hacer dibujos vectoriales, en realidad están escribiendo **código XML en formato SVG** (`<svg>`, `<rect>`, `<circle>`, `<path>`).

- Como el modelo "piensa" en texto y coordenadas matemáticas sin ver realmente el lienzo, las proporciones suelen salir chuecas, los ojos desalineados y los trazos muy básicos.
- Eso produce de serie el **efecto Paint / meme cutre** sin que tengas que forzarlo demasiado. Además, un archivo SVG ocupa apenas **1 o 2 KB**, se escala a cualquier resolución sin pixelarse y cualquier motor de interfaz (Web, Flutter, React Native, Android, Electron) lo renderiza al vuelo.

2. ¿Entrenar un modelo o usar lo que ya hay?

Tienes tres caminos según cómo quieras que funcione tu app:

Enfoque	Cómo funciona	Pros	Contras
1. Prompting (API o Modelo Local)	Le pasas el nombre del archivo (ej. `Batman_Año_Uno_01.cbr`) a un modelo rápido (Llama 3.2 3B, Qwen 2.5 Coder 3B o Haiku/Flash) y le pides solo el bloque `<svg>`.	Cero entrenamiento. Resultados desternillantes al instante.	Requiere conexión (si es API) o ~2 GB de RAM/VRAM (si es local con Ollama/Llama.cpp).
2. Fine-Tuning (LoRA) de un modelo pequeño	Entrenas un modelo diminuto (1B - 3B parámetros) con un dataset de 200-500 SVGs cutres hechos a mano o filtrados.	El modelo aprende exactamente tu estilo de "Paint" (paleta de 16 colores de Windows 95, trazos gordos, Comic Sans).	Lleva un par de tardes preparar el dataset y entrenar en Google Colab / Unsloth.
3. LLM genera	En vez de SVG puro, o además de él, pasas el SVG	Rough.js convierte líneas rectas y	Añade una dependencia visual,

Enfoque	Cómo funciona	Pros	Contras
JSON + Librería de trazo tembloroso	por una librería como **Rough.js**.	círculos perfectos en trazos chuecos pintados a mano/ratón automáticamente.	aunque es ultraligera.

3. El "Combo Perfecto" para el estilo MS Paint

Si quieres que parezca MS Paint auténtico de los 90 y no solo un diagrama aburrido de círculos perfectos, puedes combinar el **System Prompt** con un par de restricciones técnicas:

1. **Paleta Windows 95:** Obliga al modelo en el prompt a usar solo los 16 colores clásicos de Paint (`#000000`, `#FF0000`, `#FFFF00`, `#00FF00`, `#00FFFF`, `#0000FF`, `#FF00FF`, etc.).
2. **Prohibir curvas elegantes:** En el prompt, dile que en lugar de usar `<circle>` o `<rect>` limpios, use `<polyline>` o `<path>` con coordenadas ligeramente irregulares y bordes negros gruesos (`stroke-width="4"`).
3. **Tipografía Comic Sans / Pixel:** Incluye dentro del SVG una etiqueta `<text>` con `font-family="Comic Sans MS, cursive"` con el título del cómic mal centrado o torcido (`transform="rotate(-5)"`).

Ejemplo de System Prompt para probar hoy mismo:

> *"Eres un artista pésimo usando MS Paint en Windows 95 con un ratón de bola roto. Tu única tarea es generar una carátula placeholder en código SVG válido (`viewBox="0 0 300 450"`) basada en el título del cómic que te dé el usuario. Usa colores chillones básicos, figuras desproporcionadas, caras asimétricas ridículas, trazos negros gruesos (`stroke-width="4"`) y pon el título en Comic Sans torcido. Devuelve ÚNICAMENTE el código `<svg>...</svg>`."*

4. Alternativa si el dispositivo no puede correr un LLM en vivo

Si tu lector de CBR va a correr en un móvil o tablet donde meter un LLM local de 2 GB solo para las carátulas es matar moscas a cañonazos, tienes dos opciones muy limpias:

- **Pre-generar un pack de 100 placeholders temáticos:** Usas un LLM en tu PC para generar 100 SVGs cutres categorizados por palabras clave del nombre del archivo (*superhéroe, manga, espada, espacio, detective, terror, romance, default*) y los metes en la app (100 SVGs son menos de 200 KB en total).
- **Modo híbrido ("Sorpréndeme"):** Viene con el pack por defecto, pero en ajustes el usuario puede poner su API Key gratuita de Gemini Flash / Groq o conectar con su Ollama local para que cada vez que importe un `.cbr` sin portada, se cocine un dibujo único leyendo el nombre del archivo.

¿En qué lenguaje o framework estás montando el lector de CBR? Si quieres, te saco un ejemplo del código SVG que escupe este prompt con algún cómic mítico para que veas la pinta que tiene.

User prompt: Que es más rápido a la hora de hacer una nueva carátula?

Response: En términos de **velocidad pura en el momento de abrir o importar un cómic** (latencia en tiempo de ejecución), el ganador absoluto es un **generador procedural/combinatorio**, seguido por **APIs ultrarrápidas (como Groq o Gemini Flash)**.

Aquí tienes la comparativa exacta de lo que tarda cada opción en escupir una carátula nueva:

Método	Tiempo por carátula	Cómo funciona en la práctica
1. "Frankenstein" de piezas SVG (Local sin IA en vivo)	**1 a 5 milisegundos** (Instantáneo)	Usas un LLM *antes* para crear 50 fondos, 50 cuerpos, 50 cabezas y 50 bocadillos cutres. Tu app lee el nombre del `.cbr`, saca un *hash* y ensambla una portada única al vuelo + Rough.js.
2. LLM en la nube (Groq / Cerebras / Gemini Flash)	**0.4 a 1.2 segundos**	Un SVG de una portada son unos 300–500 tokens. En Groq (corriendo Llama 3 a ~800 tokens/s) la portada se dibuja casi en tiempo real.
3. LLM Local pequeño (1B - 3B en GPU/NPU)	**1.5 a 4 segundos**	Usando `llama.cpp` o `Ollama` con un modelo cuantizado (ej. Qwen 2.5 1.5B o Llama 3.2 3B). Si el usuario importa 50 cómics de golpe, tardará un par de minutos en hacer la cola.
4. LLM Local en CPU (Móvil gama media / PC sin gráfica)	**5 a 15 segundos**	Demasiado lento si el usuario mete una carpeta entera de golpe; tendrías que dejarlo generando en un hilo en segundo plano (*background worker*).

El veredicto según lo que busques

- **Si buscas velocidad de carga instantánea (0 esperas al importar una biblioteca de 200 CBRs):** Usa el **Método 1 (Frankenstein SVG)**. Le pides hoy mismo a Claude o a mí que te generemos un JSON con 30 caras mal dibujadas, 30 accesorios (espadas chuecas, capas de superhéroe, gorros de detective, pelos de anime puntiagudos) y 20 fondos de Paint. Cuando entra `Naruto_01.cbr`, tu código detecta palabras clave o tira un dado, junta 3 piezas SVG, estampa el título `"NARUTO 01"` en Comic Sans torcido y tienes millones de combinaciones en **2 milisegundos** sin gastar batería ni datos.
- **Si quieres que el chiste sea 100% específico al título del cómic (ej. que si el archivo se llama *Spiderman vs Hulk*, dibuje un monigote rojo y uno verde pegándose):** Tira por una **API rápida (Método 2)** en segundo plano. Mientras tarda ese **1 segundo** en llegar el SVG del LLM, muestras un icono de "Pintando en MS Paint..." con un reloj de arena de Windows 95, y en cuanto responde la API, guardas el `.svg` en la caché local para no volver a generarlo nunca más.

¿Prefieres que la app funcione **100% offline sin depender de servidores**, o no te importa hacer llamadas a una API cuando el usuario importe cómics?

User prompt: Entonces un modelo entrenado por mi no es viable?

Response: **Sí es totalmente viable**, y de hecho tiene una ventaja brutal: **si lo entrenas tú, puedes hacer que sea mucho más rápido y ligero que un modelo genérico.**

El problema de los modelos normales (incluso los pequeños) es que "hablan demasiado" o usan código SVG muy largo. Si entrenas el tuyo propio (haciendo *fine-tuning*), puedes enseñarle a escupir **SVGs ultracompactos** sin decir ni "Hola", bajando el tiempo de generación a **menos de 1 segundo en local**.

Por qué un modelo entrenado por ti SÍ funciona (y cómo hacerlo rápido)

1. **Tamaño microscópico (0.5B a 1.5B parámetros):** No necesitas un modelo que sepa historia o matemáticas; solo quieres que traduzca `"Título de cómic" -> "<svg>... </svg>"`. Puedes coger **Qwen 2.5 Coder 0.5B** o **Llama 3.2 1B**. Una vez cuantizado (en formato GGUF a 4 bits), el modelo entero pesa **entre 350 MB y 700 MB**. Cabe en cualquier móvil o PC sin despeinarse.

2. **Ahorro de tokens = Velocidad x3:** Un modelo normal gasta 500 tokens en hacer un dibujo. Si tú entrenas tu modelo con ejemplos de SVGs muy cortos (de solo 100-150 tokens usando `<path>` y `<circle>` directos), el modelo terminará el dibujo en **0.3 - 0.8 segundos**.

3. **Cero *System Prompt* largo:** Como el modelo ya lleva el "estilo MS Paint cutre" grabado a fuego en sus pesos (gracias a tu entrenamiento), no tienes que meterle un prompt de 200 palabras cada vez. Le pasas literalmente `Batman #1` y él ya sabe qué hacer.

La receta para entrenarlo en un fin de semana (Coste: 0 €)

No tienes que entrenar desde cero (eso cuesta millones), sino hacer un **Fine-Tuning con LoRA** usando un dataset sintético:

1. **Crear el Dataset (1-2 horas):** Usas la API de Claude o Gemini para generar automáticamente **500 ejemplos** de pares `Título -> SVG cutre estilo Paint`. Revisas por encima que sean graciosos y que el código SVG sea corto.

2. **Entrenar en Google Colab gratis (30-45 minutos):** Usas **Unsloth** (una librería gratuita de Python para entrenar LLMs el doble de rápido y gastando poca memoria). Cargas `Qwen2.5-0.5B-Instruct`, le metes tus 500 ejemplos y en media hora tienes tu modelo entrenado.

3. **Exportar a un archivo `.gguf` (5 minutos):** El propio Unsloth tiene un botón para exportarlo a formato `GGUF` (4-bit quantization). Te descargas un único archivo de ~400 MB que puedes empaquetar dentro de tu lector de cómics o descargar la primera vez que se abre la app.

¿Cuándo te compensa entrenarlo a ti?

- **Te compensa** si te apetece cacharrear y aprender cómo se entrena e integra un LLM local, y quieres poder decir *"Mi lector de cómics lleva una IA propia entrenada por mí solo para dibujar portadas cutres en Paint"*. Como reclamo para el proyecto o portfolio es un 10/10.

- **Cómo solucionar la espera al importar muchos cómics:** Aunque tarde 1 segundo por portada, en tu app pones una **cola en segundo plano**. El usuario importa 50 archivos

`.cbr`, ve un icono temporal de Paint y, mientras va leyendo el primer cómic, tu modelo va escupiendo las otras 49 portadas una por segundo en silencio.
